

the original Data Encryption Standard (DES) to address its vulnerability to brute-force attacks (where every possible key is systematically tried) due to its short 56-bit key (removing the 8 parity bits reduces the original 64-bit key to a 56-bit key). Triple DES applies the DES algorithm three times sequentially, using either two or three independent 56-bit keys, effectively increasing the key length to 112 or 168 bits. This significantly enhances security compared to DES, making brute-force attacks much more difficult.

Now, let us explore the implementation of Triple DES using the library 'Cryptography'. In the previous article, we installed and began using SageMath on our computer instead of using the online tool CoCalc. We worked with the enhanced IPython-based interface to execute SageMath code in the terminal and learned how to run SageMath programs using the command *sage*. Similarly, in this article, we will execute the SageMath program locally on our machine rather than using CoCalc.

Now, consider the SageMath program shown below, which implements Triple DES using the library Cryptography. Notice that this is the 25th program we are discussing in this series; thus, the program is aptly named 'tripleDES25.sage' (line numbers are included for clarity).

```

1.  from cryptography.hazmat.primitives.ciphers import
    Cipher, algorithms, modes
2.  from cryptography.hazmat.primitives import padding
3.  from cryptography.hazmat.backends import default_backend

4.  key = b"RAINBOWS"
5.  plaintext = "HAPPINESSISABUTTERFLY"
6.  print("Plaintext:", plaintext)

7.  padder = padding.PKCS7(64).padder( )
8.  plaintext_padded = padder.update(plaintext.encode( )) +
    padder.finalize( )
9.  cipher = Cipher(algorithms.TripleDES(key), modes.ECB( ),
    backend=default_backend( ))
10. encryptor = cipher.encryptor( )
11. ciphertext = encryptor.update(plaintext_padded) +
    encryptor.finalize( )
12. print("Ciphertext:", ciphertext.hex( ))

13. decryptor = cipher.decryptor( )
14. decrypted_padded = decryptor.update(ciphertext) +
    decryptor.finalize( )
15. unpadder = padding.PKCS7(64).unpadder( )
16. decrypted_message = unpadder.update(decrypted_padded) +
    unpadder.finalize( )
17. print("Decrypted Text:", decrypted_message.decode( ))

```

Let us go through the code line by line to understand how it works. Line 1 imports the modules *Cipher* (used to create

encryption and decryption objects), *algorithms* (provides different encryption algorithms like AES, TripleDES, etc), and *modes* (defines encryption modes like ECB, CBC, etc). Line 2 imports the module *padding* which provides functions for PKCS7 padding. This ensures that data size is a multiple of the block size. Line 3 imports the module *default_backend* which specifies the cryptographic backend required to create the *Cipher* object. Line 4 defines the encryption key as a byte string (indicated by *b*). Line 5 defines the plaintext message that will be encrypted. Line 6 prints the original plaintext before encryption. Line 7 creates a PKCS7 padder with a block size of 64 bits (8 bytes). Line 8 converts the plaintext into bytes using the function *encode()* and applies padding by using the functions *padder.update()* and *padder.finalize()*, ensuring it matches the block size. Line 9 specifies TripleDES as the encryption algorithm, Electronic Codebook (ECB) as the mode, and also specifies the cryptographic backend. Line 10 creates an *encryptor* object from the cipher. Line 11 encrypts the padded plaintext by processing it in chunks. Line 12 prints the encrypted text as a hexadecimal string for readability. Line 13 creates a *decryptor* object from the cipher. Line 14 decrypts the ciphertext, but it still includes padding. Line 15 creates an *unpadder* object to remove PKCS7 padding. Line 16 removes the padding from the decrypted data. Finally, Line 17 decodes the decrypted bytes back into a human-readable string and prints the original plaintext.

Now, let us execute the SageMath program 'tripleDES25.sage'. The program can be executed by running the command *sage tripleDES25.sage* in the terminal. Figure 1 shows the execution of the SageMath program 'tripleDES25.sage'. It can be observed in the figure that the ciphertext generated is the same as in the previous implementation of DES using 'PyCryptodome' when the plaintext and the secret key are identical. This is because we are still using an 8-byte key, effectively reducing the security of the algorithm to that of standard DES. Remember, TripleDES requires a 16-byte or 24-byte key to provide full cryptographic strength.

```

# sage tripleDES25.sage
Plaintext: HAPPINESSISABUTTERFLY
Ciphertext: fffb961a5099b22543c7adf47c8c5ee061d52ff30cc349be
Decrypted Text: HAPPINESSISABUTTERFLY

```

Figure 1: Execution of the SageMath program *tripleDES25.sage*

Now, let us familiarise ourselves with two fundamental principles in cryptography—confusion and diffusion—using the SageMath program 'tripleDES25.sage'. These concepts were introduced by Claude Shannon, a pioneer in communication and cryptography.

'Confusion' ensures that the relationship between the plaintext, ciphertext, and secret key is complex and unpredictable. This means that even a small change in the key significantly alters the ciphertext. To observe confusion